

Requirements Table

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-001	Functional	The system shall parse OpenAPI 3.0 schema definitions and generate dynamic HTTP mock endpoints returning randomized schema-compliant JSON payloads.	High	Pass: Mock endpoint responds with correct JSON schema types. Fail: Invalid endpoint path returns 200 OK.	Core value proposition of the tool; without this, no mock endpoints exist to test against.
FR-002	Functional	The system shall allow the API Developer to configure the response delay for mock endpoints.	Medium	Pass: The configured delay is applied before sending the response.	Helps simulate slow network or server responses.
FR-003	Functional	The system shall validate incoming requests based on the API specification.	High	Pass: Valid requests are accepted and invalid requests return an error.	Ensures that the mock API behaves similar to the actual API.
FR-004	Functional	The system shall allow the API Developer to configure the response delay for mock endpoints.	Medium	Pass: The configured delay is applied before sending the response.	Helps simulate slow network or server responses.
FR-005	Functional	The system shall log every mock request (endpoint, timestamp, status code, latency) and allow the API Developer to export the logs as a CSV file.	Low	Pass: The exported CSV contains one row per logged request with all required fields matching the displayed logs. Fail: The exported file is empty or omits requests.	Supports auditing, debugging, and reporting of mock server usage across a test cycle.
NFR-001	Nonfunctional	The mock server engine must sustain 1,000 mock API requests per second with user-configured simulated latency accurate within ± 10 ms.	High	Pass: Benchmarking tests confirm target throughput and latency accuracy under simulated peak load, meeting security standards. Fail: Throughput or latency accuracy falls below target under load.	Ensures the tool is usable for realistic load and integration testing, not just single-request demos.
NFR-002	Nonfunctional	The system shall provide clear error messages for invalid specifications and requests.	Medium	Pass: Error messages clearly indicate the problem when an invalid specification or request is tested. Fail: The system shows an unclear error message or does not provide any error message when an invalid specification or request is detected.	Makes the system easier for developers and QA engineers to use.